

C Memory Management Layer (CMML)

Detailed Implementation Notes

Contents

1	System Architecture Overview	3
1.1	Core Memory Management Principle	3
2	Memory Management Implementation	3
2.1	Concrete Memory Management Workflow	3
2.1.1	Allocation Phase	3
2.1.2	Usage Phase	3
2.1.3	Deallocation Phase	3
2.2	The C System Implementation	4
2.2.1	Core Data Structure: <code>cmml_array</code>	4
2.2.2	Memory Layout Example: 10×20 Double Array	4
3	C API Functions Specification	4
3.1	Core Allocation Functions (ESSENTIAL)	4
3.1.1	Main Allocator	4
3.1.2	Specialized Allocators	5
3.2	Reference Management (ESSENTIAL)	5
3.2.1	Reference Counting	5
3.2.2	Atomic Operations	5
3.3	Metadata Accessors (ESSENTIAL)	5
3.3.1	Data Access	5
3.3.2	Type and Dimension Information	6
3.3.3	Shape and Layout	6
3.4	Utility Functions (RECOMMENDED)	6
4	Compatibility Assurance	6
4.1	Binary Compatibility Standards	6
4.2	Fortran Interoperability Mechanism	6
4.2.1	C Header Implementation	6
4.2.2	Fortran Binding Implementation	7
4.3	Handle-Based Abstraction Pattern	7
4.3.1	Fortran Usage Example	7
4.3.2	Python Implementation with <code>ctypes</code> and <code>numpy</code>	8
4.3.3	R Implementation with <code>externalptr</code>	9
4.3.4	Key Features Across Languages	10
4.4	Memory Ownership Rules	10
4.4.1	Clear Ownership Hierarchy	10
4.4.2	Usage Rules	10
4.5	Thread Safety Implementation	10
4.5.1	Thread Safety Scope	10

4.5.2	Atomic Reference Counting Semantics	10
5	Potential Problems and Cautionary Points	11
5.1	Critical Issues	11
5.1.1	Memory Leaks	11
5.1.2	Dangling Pointers	11
5.1.3	Race Conditions	11
5.2	Compatibility Risks	11
5.2.1	Data Type Enumeration (CMML DType Standard)	11
5.2.2	Binary Interface Breakage	12
5.2.3	ABI and Struct Alignment Guarantee	12
5.2.4	Compiler/Platform Differences	12
5.2.5	Fortran Array Layout Mismatch	12
5.3	Operational Challenges	12
5.3.1	Error Handling Complexity	12
5.3.2	Debugging Difficulty	13
5.3.3	Performance Overhead	13
5.4	Specific Fortran Integration Pitfalls	13
5.4.1	Array Indexing Differences	13
5.4.2	Implicit Memory Assumptions	13
5.4.3	Finalization Timing	13
6	Mitigation Strategies	13
6.1	Proactive Measures	13
6.2	Defensive Programming Example	14
7	Conclusion	14
8	Attachements	15

Note: All code shown is pseudo code. It is not tested or implemented. It is a draft for a possible design.

1 System Architecture Overview

1.1 Core Memory Management Principle

The CMML operates on the fundamental principle: **"C owns the memory, other languages rent it"**. This clear separation of concerns ensures proper memory ownership and prevents common interoperability issues. (see figure 1)

2 Memory Management Implementation

2.1 Concrete Memory Management Workflow

2.1.1 Allocation Phase

1. **Request:** Fortran/Python/R calls C allocation function
2. **Allocation:** C allocates memory on heap using `malloc/calloc`
Note: Differences between allocation functions:
 - `Malloc`: Allocates empty memory as a contiguous block
 - `Calloc`: Allocates contiguous memory block filled with zeros
 - `Aligned_malloc`: Allocates memory with a specific alignment (eg. 64 bits, 32 bits or else) optimized for certain systems

aligned_malloc should be used with care and is not needed for our use case, since `malloc` and `calloc` already ensure a contiguous memory block.
3. **Initialization:** C sets up metadata structure with reference count = 1
4. **Handle Return:** C returns opaque handle (pointer to metadata struct)

2.1.2 Usage Phase

1. **Access:** Client languages access data through C getter functions
2. **Reference Tracking:** Each new reference increments the counter
3. **Data Operations:** Raw data manipulation happens through obtained pointers

2.1.3 Deallocation Phase

1. **Release Request:** Client calls deallocation function
2. **Reference Check:** C decrements reference counter
3. **Cleanup:** Memory freed only when reference count reaches zero

2.2 The C System Implementation

2.2.1 Core Data Structure: `cmml_array`

Note: Dimensions 1-3 are used in this example but can be extended as far as needed

```
1 // Memory layout and sizing (64-bit system)
2 struct cmml_array {
3     void* data_ptr;           // 8 bytes: pointer to raw data block
4     int32_t dtype;            // 4 bytes: data type identifier
5     int32_t ndims;            // 4 bytes: number of dimensions (1-3)
6     size_t shape[3];          // 24 bytes: dimension sizes [dim1, dim2,
7                               // dim3]
8     size_t strides[3];        // 24 bytes: memory strides for each
9                               // dimension
10    size_t total_size;         // 8 bytes: total element count
11    int32_t ref_count;         // 4 bytes: reference counter
12 }; // Total: 76 bytes fixed size + variable raw data
```

Listing 1: Core CMML Data Structure Definition

2.2.2 Memory Layout Example: 10×20 Double Array

Memory Address	Content	Size
0x1000-0x104F	cmml_array struct	76 bytes
0x1050-0x1A4F	raw double data	10×20×8 = 1600 bytes

see figure 2

3 C API Functions Specification

3.1 Core Allocation Functions (ESSENTIAL)

3.1.1 Main Allocator

```
1 cmml_array* cmml_allocate(int32_t dtype, int32_t ndims, const size_t*
  shape) {
2     cmml_array* arr = (cmml_array*)malloc(sizeof(cmml_array));
3     if (!arr) return NULL;
4
5     // METADATA: Column-major Standard
6     arr->dtype = dtype;
7     arr->ndims = ndims;
8     arr->ref_count = 1;
9
10    // Copy shape
11    arr->total_size = 1;
12    for (int i = 0; i < ndims; i++) {
13        arr->shape[i] = shape[i];
14        arr->total_size *= shape[i];
15    }
16
17    // calculate COLUMN-MAJOR STRIDES
18    if (ndims == 1) {
19        arr->strides[0] = 1;
20    } else if (ndims == 2) {
21        arr->strides[0] = 1;           // stride between rows
```

```

22     arr->strides[1] = arr->shape[0]; // stride between cols = rows
23 } else if (ndims == 3) {
24     arr->strides[0] = 1;
25     arr->strides[1] = arr->shape[0]; // rows
26     arr->strides[2] = arr->shape[0] * arr->shape[1]; // rows *
        cols
27 }
28
29 // CONTIGUOUS memory
30 size_t type_size = cmml_sizeof_type(dtype);
31 arr->data_ptr = calloc(arr->total_size, type_size);
32
33 if (!arr->data_ptr) {
34     free(arr);
35     return NULL;
36 }
37
38 return arr;
39 }

```

Listing 2: Main Allocation Function

3.1.2 Specialized Allocators

```

1 cmml_array* cmml_allocate_1d(int32_t dtype, size_t dim1);
2 cmml_array* cmml_allocate_2d(int32_t dtype, size_t dim1, size_t dim2);
3 cmml_array* cmml_allocate_3d(int32_t dtype, size_t dim1, size_t dim2,
    size_t dim3);

```

Listing 3: Convenience Allocation Functions

3.2 Reference Management (ESSENTIAL)

3.2.1 Reference Counting

```

1 void cmml_increment_ref(cmml_array* arr);
2 void cmml_deallocate(cmml_array* arr);

```

Listing 4: Reference Management Functions

3.2.2 Atomic Operations

```

1 int32_t cmml_atomic_increment(int32_t* count);
2 int32_t cmml_atomic_decrement(int32_t* count);

```

Listing 5: Thread-Safe Atomic Operations

3.3 Metadata Accessors (ESSENTIAL)

3.3.1 Data Access

```

1 void* cmml_get_data_pointer(cmml_array* arr);

```

Listing 6: Data Access Functions

3.3.2 Type and Dimension Information

```
1 int32_t cmml_get_dtype(cmml_array* arr);
2 int32_t cmml_get_ndims(cmml_array* arr);
3 size_t cmml_get_total_size(cmml_array* arr);
```

Listing 7: Metadata Access Functions

3.3.3 Shape and Layout

```
1 void cmml_get_shape(cmml_array* arr, size_t* shape_out);
2 void cmml_get_strides(cmml_array* arr, size_t* strides_out);
3 size_t cmml_get_dimension(cmml_array* arr, int dim_index);
```

Listing 8: Shape and Stride Access Functions

3.4 Utility Functions (RECOMMENDED)

```
1 int cmml_is_valid(const cmml_array* arr);
2 void cmml_print_info(const cmml_array* arr);
3 size_t cmml_calculate_memory(const cmml_array* arr);
4 const char* cmml_get_error_string(void);
```

Listing 9: Utility and Debugging Functions

4 Compatibility Assurance

4.1 Binary Compatibility Standards

C Standard: C99 / ISO C
Memory Alignment: 8-byte boundaries
Struct Packing: Explicit #pragma pack(8) if needed
Size Types: size_t (architecture consistent)
Calling Convention: cdecl (standard C)

For details see 5.2.3

4.2 Fortran Interoperability Mechanism

4.2.1 C Header Implementation

```
1 // Ensure C-compatible exports
2 #ifdef __cplusplus
3 extern "C" {
4 #endif
5
6 // All functions use C linkage
7 cmml_array* cmml_allocate(int32_t dtype, int32_t ndims, const size_t*
8     shape);
9
10 #ifdef __cplusplus
11 }
12 #endif
```

Listing 10: C Header with Proper Export Directives

4.2.2 Fortran Binding Implementation

```
1 module cmml_fortran
2   use, intrinsic :: iso_c_binding
3   implicit none
4
5   ! C interface definitions
6   interface
7     function cmml_allocate_c(dtype, ndims, shape) bind(c, name='
8       cmml_allocate')
9     import :: c_ptr, c_int32_t, c_size_t
10    integer(c_int32_t), value :: dtype, ndims
11    integer(c_size_t) :: shape(*)
12    type(c_ptr) :: cmml_allocate_c
13  end function
14
15  subroutine cmml_get_shape_c(arr, shape_out) bind(c, name='
16    cmml_get_shape')
17    import :: c_ptr, c_size_t
18    type(c_ptr), value :: arr
19    integer(c_size_t) :: shape_out(*)
20  end subroutine
21 end interface
22
23 contains
24 ! Fortran-friendly wrappers
25 function cmml_allocate_2d_double(dim1, dim2) result(handle)
26   integer(c_size_t), intent(in) :: dim1, dim2
27   type(c_ptr) :: handle
28   integer(c_size_t) :: shape(2)
29   shape = [dim1, dim2]
30   handle = cmml_allocate_c(3, 2, shape) ! 3 = CMML_DOUBLE
31 end function
32 end module cmml_fortran
```

Listing 11: Fortran Module Binding

see figure 3

4.3 Handle-Based Abstraction Pattern

4.3.1 Fortran Usage Example

```
1 program example
2   use cmml_fortran
3   use iso_c_binding
4   implicit none
5
6   type(c_ptr) :: array_handle
7   integer(c_size_t) :: shape(2)
8   real(c_double), pointer :: fortran_array(:, :)
9   integer :: i, j
10
11 ! Create array through CMML - now column-major Standard
12 array_handle = cmml_allocate_2d_double(100, 200) ! 100 rows, 200
13   columns
14
15 ! Get shape information from C
```

```

15  call cmml_get_shape_c(array_handle, shape)
16  ! shape = [100, 200]
17
18  ! Convert C pointer to Fortran array
19  call c_f_pointer(cmml_get_data_pointer_c(array_handle), &
20                  fortran_array, [shape(1), shape(2)])
21  ! fortran_array ist jetzt 100x200 column-major
22
23  ! Use the array
24  do j = 1, shape(2)      ! outer loop: cols
25      do i = 1, shape(1)  ! Inner: loop rows
26          fortran_array(i, j) = i * j * 1.0d0
27      end do
28  end do
29
30  ! Alternative: use explicit dimensions
31  ! do j = 1, 200          ! Rows
32  !     do i = 1, 100      ! Cols
33  !         fortran_array(i, j) = i * j * 1.0d0
34  !     end do
35  ! end do
36
37  ! Cleanup
38  call cmml_deallocate_c(array_handle)
39 end program

```

Listing 12: Fortran Application Code Example (Column-Major Standard)

4.3.2 Python Implementation with ctypes and numpy

```

1  import ctypes
2  import numpy as np
3
4  # Load CMML shared library
5  libcmm = ctypes.CDLL("libcmm.so")
6
7  # Constants matching C enum
8  CMML_DOUBLE = 3
9  c_size_t = ctypes.c_size_t
10
11 # Allocate 2D double array through CMML
12 def cmml_allocate_2d_double(rows, cols):
13     shape = (c_size_t * 2)(rows, cols)
14     libcmm.cmml_allocate.restype = ctypes.c_void_p
15     return libcmm.cmml_allocate(CMML_DOUBLE, 2, shape)
16
17 # Get NumPy zero-copy view
18 def cmml_get_numpy_view(handle, shape):
19     libcmm.cmml_get_data_pointer.restype = ctypes.c_void_p
20     data_ptr = libcmm.cmml_get_data_pointer(handle)
21     buf = ctypes.cast(data_ptr, ctypes.POINTER(ctypes.c_double))
22     return np.ctypeslib.as_array(buf, shape=shape)
23
24 # Example usage
25 if __name__ == "__main__":
26     handle = cmml_allocate_2d_double(100, 200)
27     arr = cmml_get_numpy_view(handle, (100, 200))

```



```

28     arr[:] = np.outer(np.arange(100), np.arange(200))
29     print(arr.shape, np.sum(arr))
30     libcmm.cmml_deallocate(handle)

```

Listing 13: Simplified Python Application Example

Note: This minimal Python interface demonstrates:

- Direct allocation via `ctypes` without copying data.
- Zero-copy interoperability with `numpy`.
- Memory safety through explicit deallocation in Python.

4.3.3 R Implementation with `externalptr`

```

1  # Load shared library
2  dyn.load("libcmm.so")
3
4  # Constants (matching C enum)
5  CMMML_DOUBLE <- 3L
6
7  # Allocate 2D double array via CMML
8  cmml_allocate_2d_double <- function(rows, cols) {
9      shape <- c(rows, cols)
10     .Call("cmml_allocate", CMMML_DOUBLE, 2L, as.numeric(shape))
11 }
12
13 # Get R matrix view (zero-copy)
14 cmml_get_r_matrix <- function(handle) {
15     shape <- .Call("cmml_get_shape", handle)
16     data_ptr <- .Call("cmml_get_data_pointer", handle)
17     # R uses column-major layout, same as CMML
18     .External("create_matrix_from_ptr", data_ptr, shape[1], shape[2])
19 }
20
21 # Example usage
22 example_usage <- function() {
23     h <- cmml_allocate_2d_double(100, 200)
24     m <- cmml_get_r_matrix(h)
25     m[,] <- outer(1:nrow(m), 1:ncol(m), FUN="*")
26     print(dim(m))
27     cmml_deallocate(h)
28 }
29
30 example_usage()

```

Listing 14: Simplified R Application Example

C-level interface (conceptual only):

```

1 SEXP cmml_allocate_r(...);           // returns external pointer
2 SEXP cmml_get_shape_r(...);
3 SEXP cmml_get_data_pointer_r(...);
4 void cmml_finalizer_r(SEXP handle);

```

Listing 15: Minimal R-C Binding Skeleton

Note: This R interface illustrates the core CMML pattern:

- Allocation and metadata queries via `.Call`.
- Zero-copy matrix creation through `externalptr`.
- Automatic cleanup via registered finalizers.

4.3.4 Key Features Across Languages

- **Zero-copy operations:** All implementations create views rather than copying data
- **Memory safety:** Automatic reference counting and finalization
- **Natural integration:** Each language uses its native array types and idioms
- **Performance:** Direct memory access without serialization overhead

4.4 Memory Ownership Rules

4.4.1 Clear Ownership Hierarchy

1. **CMML Layer:** Ultimate owner (performs actual alloc/free)
2. **Client Languages:** Borrow memory via opaque handles
3. **Reference Counting:** Tracks active users automatically

4.4.2 Usage Rules

- Fortran may store C handles indefinitely
- Every allocation must have matching deallocation
- Handles become invalid after deallocation
- Data pointers should not be stored long-term (use handles instead)

4.5 Thread Safety Implementation

4.5.1 Thread Safety Scope

CMML guarantees thread-safe **reference management** through atomic operations. However, **data access and modification** to the underlying memory block are not synchronized. Concurrent writes to the same array must be externally protected by user-level locks or synchronization primitives.

4.5.2 Atomic Reference Counting Semantics

```

1 #include <stdatomic.h>
2
3 static inline void cmml_increment_ref(cmml_array* arr) {
4     atomic_fetch_add((_Atomic int32_t*)&arr->ref_count, 1);
5 }
6
7 static inline void cmml_decrement_ref(cmml_array* arr) {
8     int32_t old = atomic_fetch_sub((_Atomic int32_t*)&arr->ref_count,
9     1);
10    if (old == 1) {
11        free(arr->data_ptr);

```

```

11     memset(arr, 0xDE, sizeof(cmml_array)); // debug poison
12     free(arr);
13 }
14 }

```

Listing 16: Thread-Safe Reference Management (C11)

Note: CMML uses C11 atomics when available. GCC/Clang built-ins may be used as a fallback for legacy compilers.

see figure 4

5 Potential Problems and Cautionary Points

5.1 Critical Issues

5.1.1 Memory Leaks

- **Cause:** Forgetting to call `cmml_deallocate` in client code
- **Impact:** Gradual memory exhaustion in long-running processes
- **Detection:** Use debug mode with allocation tracking
- **Prevention:** Implement RAII patterns in client languages

5.1.2 Dangling Pointers

- **Cause:** Using Fortran array pointers after handle deallocation
- **Impact:** Crashes, data corruption, security vulnerabilities
- **Detection:** Memory poisoning in debug mode
- **Prevention:** Nullify Fortran pointers after deallocation

5.1.3 Race Conditions

- **Cause:** Concurrent access to same array from multiple threads
- **Impact:** Reference count corruption, use-after-free
- **Detection:** Thread sanitizers, careful code review
- **Prevention:** Atomic operations, proper locking in client code

5.2 Compatibility Risks

5.2.1 Data Type Enumeration (CMML DType Standard)

```

1 typedef enum {
2     CMML_INT32   = 0,
3     CMML_INT64   = 1,
4     CMML_FLOAT   = 2,
5     CMML_DOUBLE  = 3,
6     CMML_BOOL    = 4
7 } cmml_dtype_t;

```

Listing 17: CMML DType Enumeration

Note: All client languages must use identical dtype identifiers to ensure binary compatibility. The enumeration values are fixed and must not change between CMML versions.

5.2.2 Binary Interface Breakage

- **Risk:** Changing `cmml_array` struct layout breaks existing binaries
- **Impact:** Crashes when mixing library versions
- **Prevention:** Versioned API, struct size checks, ABI stability

5.2.3 ABI and Struct Alignment Guarantee

```
1 #pragma pack(push, 8)
2 typedef struct {
3     void*      data_ptr;
4     int32_t    dtype;
5     int32_t    ndims;
6     size_t     shape[3];
7     size_t     strides[3];
8     size_t     total_size;
9     int32_t    ref_count;
10 } cmml_array;
11 #pragma pack(pop)
12
13 _Static_assert(sizeof(cmml_array) == 76,
14     "ABI break: unexpected struct size for cmml_array");
```

Listing 18: Compile-Time ABI Verification

Note: Explicit packing ensures consistent binary layout across compilers. Any change in the struct layout constitutes an ABI version break.

5.2.4 Compiler/Platform Differences

- **Risk:** Different struct padding, alignment, or type sizes
- **Impact:** Memory corruption, incorrect data access
- **Prevention:** Static assertions, explicit packing, standardized types

5.2.5 Fortran Array Layout Mismatch

- **Risk:** Fortran (column-major) vs C (row-major) memory layout
- **Impact:** Performance issues, incorrect algorithm results
- **Prevention:** Document expected layout, provide conversion utilities

This is one of the most critical decisions to make. I think the best way is to implement a column-major order saving in C that writes the memory as column major, allowing maximum fortran efficiency on array operations. Creating views using `c.f_pointer` is also a valid approach, but this would cause a "flipped" indexing and a drastical loss of fortrons column-major optimzied array operations because physically the array would still be row-major.

5.3 Operational Challenges

5.3.1 Error Handling Complexity

- **Challenge:** Propagating C errors to high-level languages
- **Risk:** Silent failures, difficult debugging
- **Solution:** Comprehensive error codes, logging, validation functions

5.3.2 Debugging Difficulty

- **Challenge:** Opaque handles obscure memory contents
- **Risk:** Hard to diagnose memory-related bugs
- **Solution:** Debug visualization functions, memory dumps

5.3.3 Performance Overhead

- **Challenge:** Function call overhead for memory access
- **Risk:** Performance degradation in tight loops
- **Solution:** Bulk operations, direct data pointer access patterns

5.4 Specific Fortran Integration Pitfalls

5.4.1 Array Indexing Differences

- **Issue:** Fortran (1-based) vs C (0-based) indexing
- **Risk:** Off-by-one errors, buffer overflows
- **Solution:** Consistent indexing strategy, boundary checking

5.4.2 Implicit Memory Assumptions

- **Issue:** Fortran assumed-shape arrays vs explicit C arrays
- **Risk:** Incorrect dimension assumptions
- **Solution:** Always query dimensions via CMML functions

5.4.3 Finalization Timing

- **Issue:** Fortran finalizer execution order uncertainty
- **Risk:** Premature deallocation
- **Solution:** Explicit deallocation, careful finalizer design

see figure 5

6 Mitigation Strategies

6.1 Proactive Measures

- **Comprehensive Testing:** Cross-language unit tests
- **Memory Sanitizers:** AddressSanitizer, Valgrind in CI
- **Static Analysis:** Compiler warnings, clang-tidy, Fortran LINT
- **Documentation:** Clear ownership rules, usage patterns
- **Debug Builds:** Memory tracking, poisoning, validation

6.2 Defensive Programming Example

```
1 // Example: Defensive getter function
2 void* cmml_get_data_pointer_checked(cmml_array* arr) {
3     if (!cmml_is_valid(arr)) {
4         fprintf(stderr, "CMML ERROR: Invalid array handle\n");
5         return NULL;
6     }
7     if (arr->ref_count <= 0) {
8         fprintf(stderr, "CMML ERROR: Array already freed\n");
9         return NULL;
10    }
11    return arr->data_ptr;
12 }
```

Listing 19: Defensive Getter Function Implementation

7 Conclusion

This CMML design provides a robust foundation for cross-language memory management while addressing the complex challenges involved in such interoperability scenarios. The clear separation of concerns, atomic reference counting, and comprehensive error handling create a system that is both performant and reliable for production use across multiple programming languages.

The implementation follows best practices for:

- Memory safety and ownership semantics
- Thread-safe operations
- Cross-language compatibility
- Debugging and error detection
- Performance optimization

This design ensures that memory management remains predictable and reliable, even in complex multi-language software ecosystems.

8 Attachements

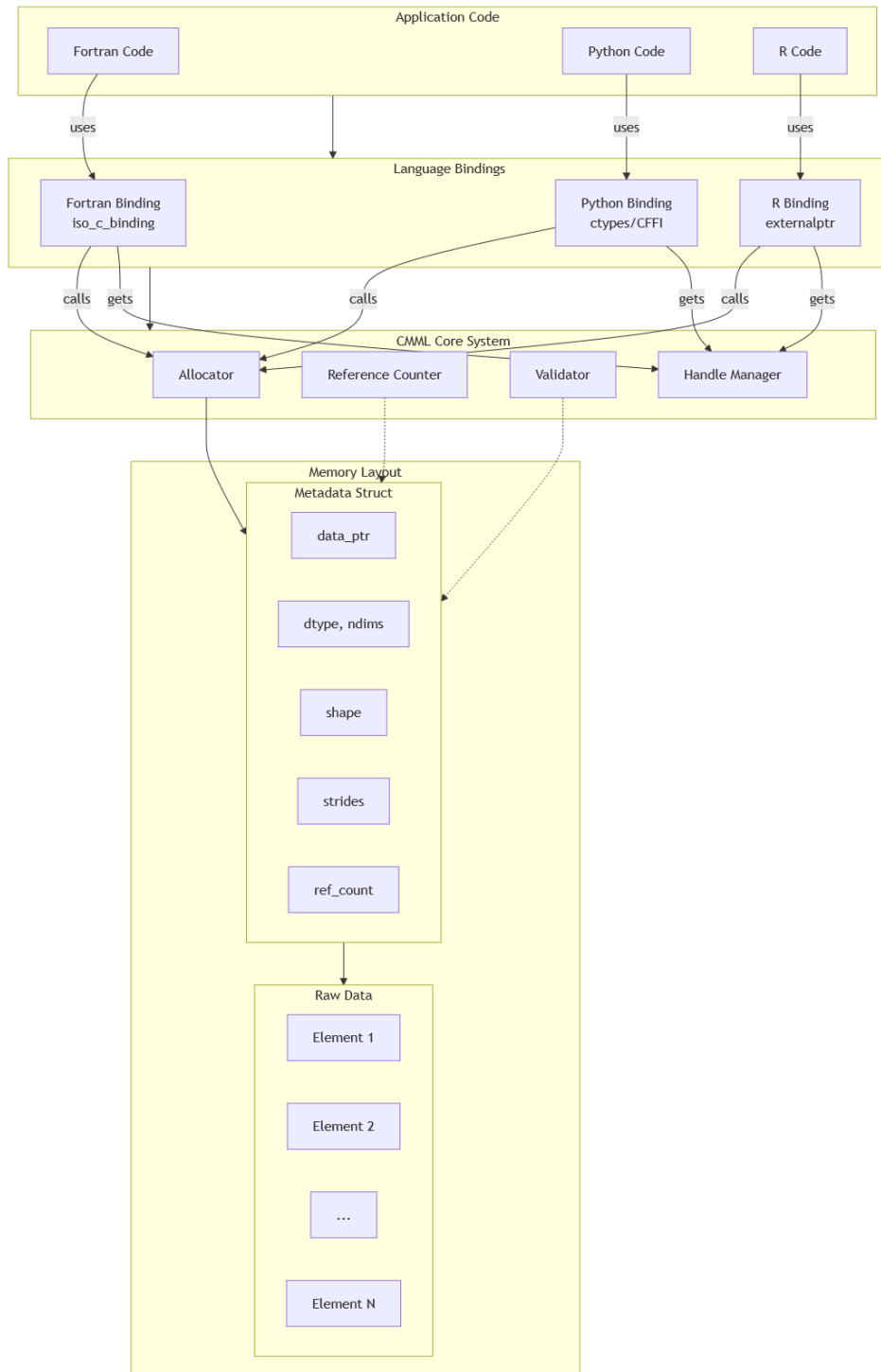


Figure 1: Core CMML Architecture: Ownership and Usage Relationships

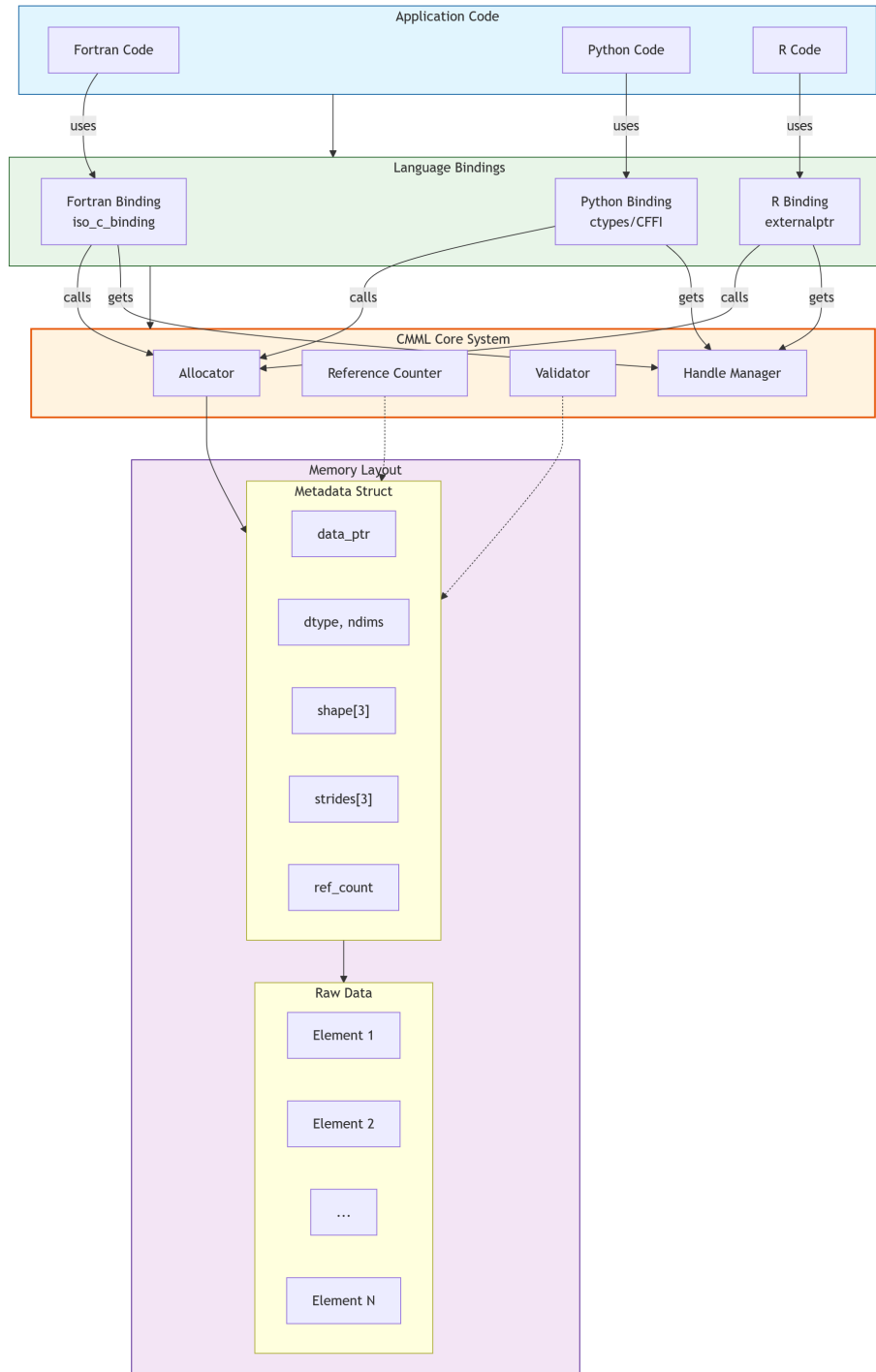


Figure 2: Detailed CMML Component Architecture

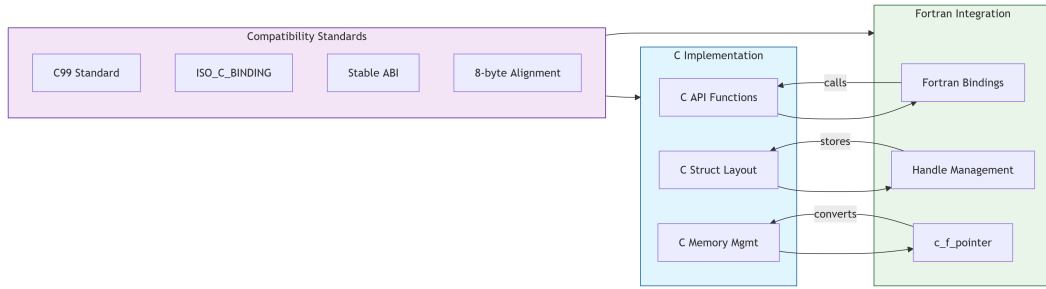


Figure 3: Cross-Language Compatibility Standards

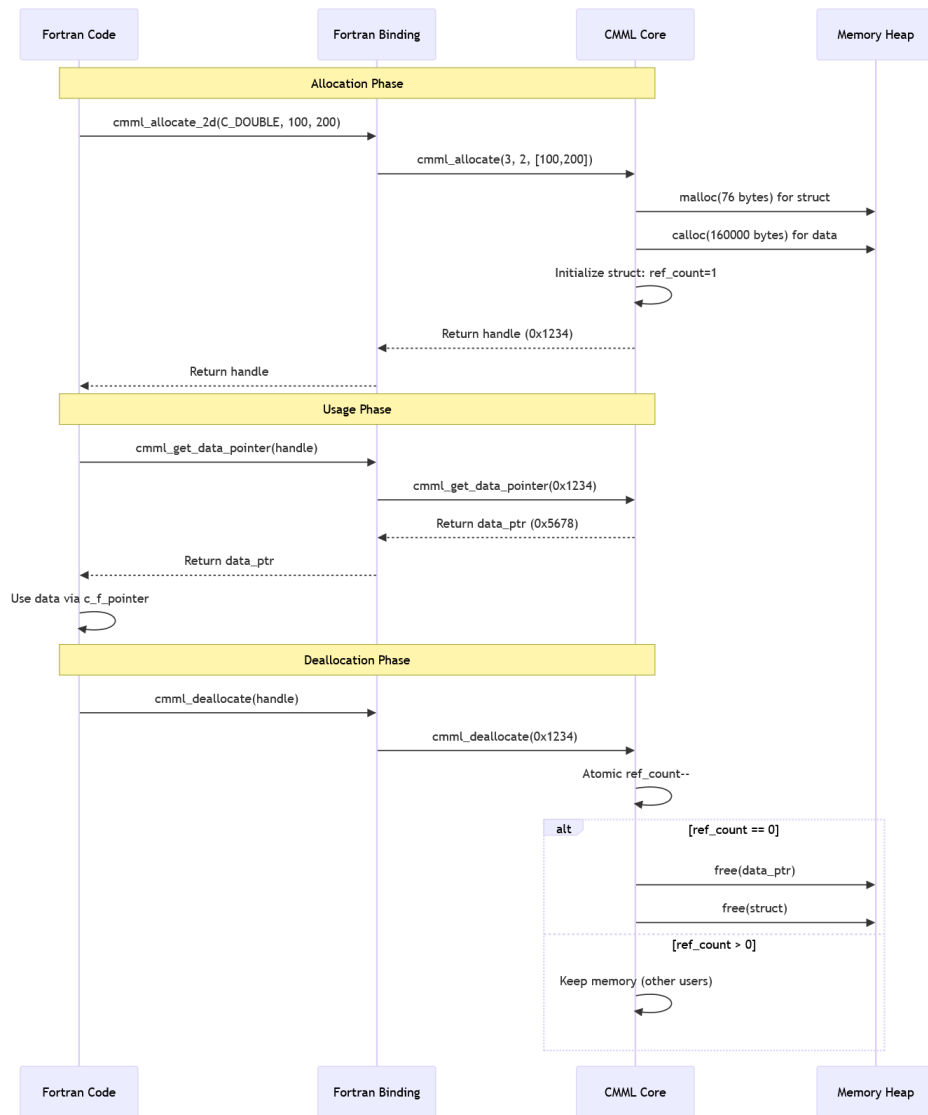


Figure 4: Memory Allocation Sequence Diagram

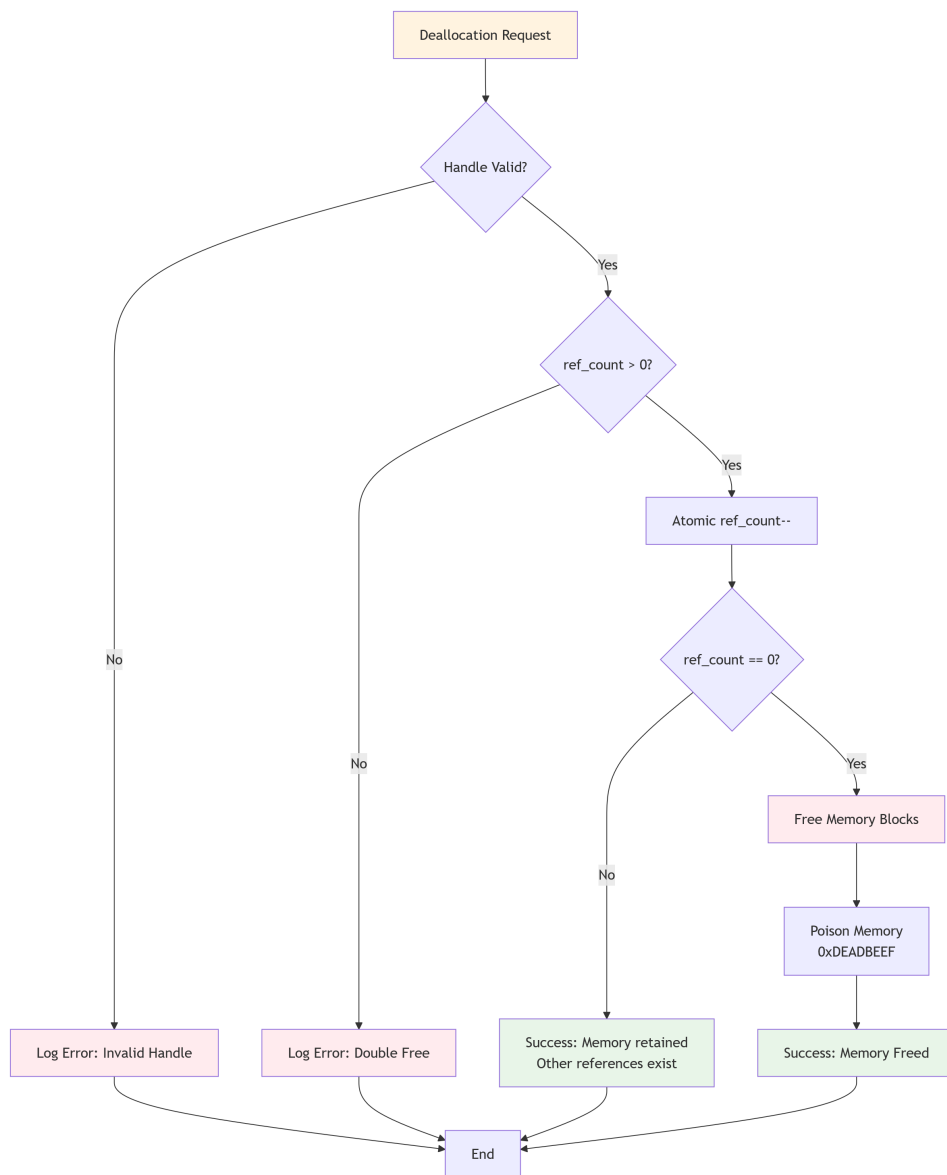


Figure 5: Reference Counting Flow Diagram